UNIVERSITAT POLITÈCNICA DE VALÈNCIA

DSIC

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Dpto. de Sistemas Informáticos y Computación

Tipos Cociente en un Lenguaje Dependientemente Tipado

Trabajo Fin de Máster

Máster Universitario en Ingeniería y Tecnología de Sistemas Software

AUTOR: Vegas, Luis

Tutor: Escobar Román, Santiago

CURSO ACADÉMICO: 2025-2026

Índice general

1. Introducción **1**
1.1. Motivación 1
1.2. Objetivos 1
1.3. Estructura del la tesis 1

2. Fundamentos teóricos **2**
2.1. El Lambda cálculo 2
2.1.1. β-reducción 3
2.1.2. Variables libres y ligadas 3
2.1.3. Sustitución 4
2.1.4. Confluencia 5
2.2. Teoría de Tipos 8
2.3. Lambda cálculo simplemente tipado 10
2.4. Tipos dependientes 10
2.5. Teoria de tipos de Martin-Löf 10
2.6. Reescritura de terminos en Maude 10
2.7. Tipos Cociente 10

3. Un lenguaje dependientemente tipado **11**
3.1. Sintaxis 11
3.2. Sustitucion 11
3.3. Normalizacion 11
3.4. Verificacion de tipos 11

4. Extension con tipos cociente **12**
4.1. Formacion 12
4.2. Introduccion 12
4.3. Eliminacion 12
4.4. Computacion 12

5. Conclusiones **13**

A. Código Maude **14**

Capítulo 1

Introducción

1.1. Motivación

1.2. Objetivos

1.3. Estructura del la tesis

Capítulo 2

Fundamentos teóricos

2.1. El Lambda cálculo

El lambda cálculo fue inventado por Alonzo Church en la decada de los 80 con el proposito de capturar la nocion abstracta de la aplicabilidad de una funcion, sin embargo, conviene precisar en qué sentido. Una función puede entenderse de dos maneras: de forma extensional, como el conjunto de pares entrada–salida que la determina, o de forma intensional, como la regla o el proceso de cómputo que, dado un argumento, prescribe cómo obtener el resultado (Geuvers, 2014). Es esta segunda lectura, la de la función como algoritmo, la que el lambda cálculo formaliza (Barendregt, 1984; Geuvers, 2014). En su origen, Church concibió el sistema como parte de una fundamentación de la lógica y las matemáticas; dicha formulación resultó ser inconsistente, tal y como evidenciaron Kleene y Rosser, por lo que Church aisló su núcleo puramente computacional, libre de constantes lógicas, dando lugar al lambda cálculo tal y como se conoce hoy (Barendregt, 1984), siendo ampliamente adoptado como un sistema formal de lógica matemática.

Este sistema ocupa, además, un lugar cenral el diseño de lenguajes de programación. De forma análoga, la manera en la que se programa en el mismo es muy similar a cómo se programa en un lenguaje funcional hoy en dia. De hecho, lenguajes como Haskell, Agda, la familia de lenguajes basados en ML, y LISP, estan ampliamente inspirados en un conjunto del lambda calculus y, pueden entenderse, en buena medida, como extensiones dotadas de azucar sintactico, sistemas de tipos y otras primitivas (Pierce, 2002).

El lambda calculo es un lenguaje muy potente y, a su vez, sumamente simple. Un término puede estar compuesto, o bien, formarse, de tres elementos: variables, aplicacion y abstracción lambda. De forma inductiva, fijado un conjunto infinito numerable de variables V, el conjunto Λ de los términos lambda se define como el menor conjunto cerrado bajo las siguientes reglas (Barendregt, 1984; Pierce, 2002):

$$
\begin{aligned}
M := \quad & x && (1)\\
\mid \quad & \lambda x.\, M && (2)\\
\mid \quad & M\ N && (3)
\end{aligned}
$$

Donde, (1) es un conjunto infinito de variables, tal que si $x, y, z \in V$ y $V \in \Lambda$, entonces $V \in \Lambda$. (2) es una abstracción lambda, si $x \in V$ y además $M \in \Lambda$, entonces $\lambda x.\, M \in \Lambda$. (3) Es una aplicación de dos términos, si $M, N \in \Lambda$ entonces $MN \in \Lambda$.

A continuación se detallan propiedades de sumo interés que articulan, posteriormente, los fundamentos del lenguaje que describe esta tésis: la distinción entre variables libres y ligadas, junto con la α-equivalencia que de ella se deriva; la operación de sustitución que evita la captura de variables; y la β-reducción como noción primaria de cómputo. Se mencionará igualmente la η-conversión, que captura la extensionalidad de las funciones.

2.1.1. β-reducción

La noción de evaluacion de un término viene dada por beta-reducción, donde la notación $\to_\beta$ es empleada. La $\to_\beta$ está basada en la *sustitución*, formalmente expresada como $M[x := N]$, significando que M es un término donde x ha sido reemplazado por N.

Por tanto, una expresión de la forma $(\lambda x.\, M)\, N$ puede ser reescrita como $M[\, x := N]$, donde toda ocurrencia de x en M (estando x libre en M), debe ser reemplazada por N (Geuvers, 2014).

Ejemplo 2.1. $(\lambda x.\, x + 1)\, 5$ reduce a $(x + 1)[x := 5] = 5 + 1 = 6$.

Ejemplo 2.2. La función identidad aplicada a un término cualquiera devuelve dicho término: $(\lambda x.\, x)\, y \to_\beta x[x := y] = y$.

Ejemplo 2.3. La β-reducción puede encadenarse, reduciendo paso a paso hasta que no queden más redexes. considérese el combinador $K \equiv \lambda x.\, \lambda y.\, x$ aplicado a dos argumentos:

$$\begin{aligned}(\lambda x.\, \lambda y.\, x)\, a\, b &\to_\beta (\lambda y.\, x)[x := a]\, b \\ &= (\lambda y.\, a)\, b \\ &\to_\beta a[y := b] = a.\end{aligned}$$

Nótese que el segundo argumento, b, es descartado, pues y no ocurre en el del lambda.

2.1.2. Variables libres y ligadas

La abstracción $\lambda x.\, M$ introduce un *binder* sobre la variable x, cuyo *alcance* es el cuerpo de M. Decimos, pues, que una ocurrencia de una variable x está *ligada* cuando aparece dentro del alcance del λx que la introduce; en caso contrario, dicha ocurrencia se dice *libre*.

Conviene resaltar que una misma variable puede ocurrir libre y ligada dentro de un mismo término. En $(\lambda x.\, x)\, x$, por ejemplo, la primera ocurrencia de x está ligada por el λx, mientras que la segunda, externa a la abstracción, permanece libre.

Formalmente, el conjunto de variables libres de un término, denotado $FV(M)$, se define de forma inductiva sobre la estructura del término (Pierce, 2002):

$$
\begin{aligned}
&(1)\ FV(x) = \{x\},\\
&(2)\ FV(\lambda x.\, M) = FV(M) \setminus \{x\},\\
&(3)\ FV(M\, N) = FV(M) \cup FV(N).
\end{aligned}
$$

(1) indica que una variable, por sí sola, es libre. (2) establece que al abstraer sobre x, esta deja de ser libre en $\lambda x.\, M$, pues queda ligada por el nuevo binder; por ello se retira del conjunto. (3) recoge que las variables libres de una aplicación son, simplemente, la unión de las libres de cada subtérmino M y N respectivamente. Un término M tal que $FV(M) = \emptyset$ se denomina *cerrado*.

Por último, el nombre concreto de una variable ligada carece de relevancia: lo único que importa es la estructura de ligaduras que establece. Así, $\lambda x.\, x$ y $\lambda y.\, y$ denotan exactamente la misma función identidad. A esta noción, que identifica términos que difieren únicamente en el nombre de sus variables ligadas, se le conoce como α-*equivalencia*, y se denota $=_\alpha$. El proceso de renombrar una variable ligada para obtener un término α-equivalente recibe el nombre de α-*conversión*, y será fundamental para definir correctamente la sustitución bajo un binder.

2.1.3. Sustitución

Anteriormente, definimos la sustitución en términos de $\rightarrow_\beta$ reducción, sin embargo, la sustitucion posee en sí reglas de computo que valen la pena considerar pues, dependiendo del término que se esté evaluando, la sustitución tendrá, o no, efecto del todo.

Se definen, pues, las reglas de computo de la sustitucion de acuerdo a lo discutido por Pierce (2002). Más formalmente:

$$
\begin{aligned}
&(1)\ x[x := N] \equiv N,\\
&(2)\ y[x := N] \equiv y \text{ if } x \not\equiv y,\\
&(3)\ (PQ)[x := N] \equiv (P[x := N])(Q[x := N]),\\
&(4)\ (\lambda y.\, P)[x := N] \equiv \lambda z.\, (P^{y\rightarrow z}[x := N]),\ \text{if } \lambda z.\, P^{y\rightarrow z} \text{ is an } \alpha\text{-variant of}\\
&\quad \lambda y.\, P \text{ such that } z \notin FV(N).
\end{aligned}
$$

Para (1), sustituyendo x, por N dentro de x, siendo x el termino que buscamos sustituir, obtenemos N. De forma analoga, sustituyendo cada ocurrencia de x en el termino y, por N, si x e y no son iguales, el termino intacto, y, sera retornado. El caso de (3), tiene sentido cuando P es una abstraccion aplicada a un termino Q. Concretamente, se reduce a la sustitucion de x por N, en ambos P y Q. (4) contempla la sustitución bajo un binder. Lo que se busca es sustituir x por N dentro del cuerpo P de la abstracción $\lambda y.\, P$. Sin embargo, si procedíeramos de forma ingenua y N contuviera a y como variable libre, dicha y quedaría inadvertidamente atrapada por

el binder λy, alterando por completo el significado del término. A este fenómeno se le conoce como *captura de variables*.

Para evitarlo, antes de sustituir se renombra la variable ligada y por una variable fresca z, mediante α-conversión, escogida de modo que $z \notin FV(N)$ (y, por extensión, que no genere conflicto con las variables ya presentes en P). Solo entonces se procede a sustituir x por N dentro del cuerpo ya renombrado. De esta manera, la sustitución preserva el significado del término original, razón por la cual se la describe como una sustitución *que evita la captura* (Pierce, 2002).

Ejemplo 2.4. Supóngase la sustitución $(\lambda y.\, x)[x := y]$. Si sustituyéramos directamente, obtendríamos $\lambda y.\, y$, es decir, la identidad: la y libre que introdujimos habría sido capturada por el binder. Renombrando primero la variable ligada a una fresca z, obtenemos $(\lambda z.\, x)[x := y] = \lambda z.\, y$, que es el resultado correcto, pues la y permanece libre.

2.1.4. Confluencia

Anteriormente señalamos cómo la β-reducción emula, en cierto sentido, el proceso de cómputo de una expresión dada: evaluado un término, aspiramos a obtener su forma normal, esto es, un término en el que ya no resta redex alguno que permita seguir reduciendo (Barendregt, 1984; Pierce, 2002).

Ahora bien, el orden en que se contraen tales redexes no es indiferente, pues puede influir en si el cómputo termina o, mejor dicho, en si alcanza una forma normal. Surge así una pregunta natural. Dado un término t del que divergen dos reducciones, $t \to_\beta^* t_1$ y $t \to_\beta^* t_2$, en uno o varios pasos, ¿es posible reconciliar ambas ramas en un término común u alcanzable tanto desde t_1 como desde t_2? Cuando tal u existe para cualquier par de reducciones divergentes, decimos que la reducción es *confluente* sobre t.

En el presente apartado describimos con detenimiento la propiedad de confluencia y exponemos por qué la β-reducción la satisface.

La confluencia, conocida también como teorema de Church-Rosser, es una propiedad de los sistemas de reescritura que caracteriza cuándo dos términos obtenidos de uno dado, por reducción en uno o más pasos, convergen a un mismo resultado. Su relevancia radica en que el resultado del cómputo queda determinado con independencia del orden en que se ejecuten los pasos de reducción (Barendregt, 1984). Formalmente:

Teorema 2.1 (Church-Rosser)**.** *Sea M un término del λ-cálculo. Si $M \to_\beta^* N_1$ y $M \to_\beta^* N_2$, entonces existe un término N_3 tal que $N_1 \to_\beta^* N_3$ y $N_2 \to_\beta^* N_3$ (Barendregt, 1984, p. 62).*

Conviene precisar que el teorema habla de la *clausura reflexivo-transitiva* de la reducción, $\to_\beta^*$, es decir, de cero o más pasos de β-reducción. Esta distinción no es superflua, pues la β-reducción no cumple la versión más fuerte de la propiedad, conocida como *propiedad del diamante* en un solo paso: si de M contraemos dos

redexes distintos en un único paso para obtener N_1 y N_2, no siempre existe un N_3 alcanzable desde ambos en un solo paso. La razón es la duplicación, pues un redex puede copiar a otro cuando el argumento contiene, a su vez, redexes, de modo que un lado de la bifurcación necesita varios pasos para reabsorber las copias que el otro lado ya había contraído. Lo que el teorema de Church-Rosser garantiza es la versión global, sobre $\rightarrow_\beta^*$, que sí se sostiene.

Ademas, por simplicidad, hasta ahora hemos equivocadamente abusado del término de confluencia. Resulta importante separar dos nociones de la misma: la confluencia local y la confluencia global. Decimos que un sistema es *localmente confluente* cuando, partiendo de M y dando un paso hacia N_1 y otro hacia N_2, existe un término común alcanzable desde ambos en cero o máas pasos. La confluencia local es más débil que la confluencia global, y la implicación de la primera a la segunda no siempre se cumple, pues solo se obtiene bajo hipótesis adicionales, como la terminación, de acuerdo al lema de Newman (aca citar o mencionar luego??). (Barendregt, 1984).

Lema 2.2. *1. M es una forma normal si M no contiene más redexes.*

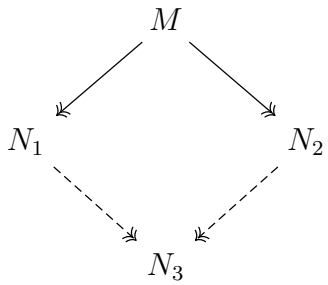
2. M tiene una forma normal si existe un N, en forma normal, tal que $M =_\beta N$. Tal N es la forma normal de M.

Una consecuencia inmediata, y quizá la más útil en la práctica, es la unicidad de la forma normal. Si un término admite forma normal, esta es única salvo α-equivalencia.

Corolario 2.3. *Si M posee una forma normal, esta es única salvo α-equivalencia. En efecto, supóngase que N_1 y N_2 son ambas formas normales de M, de modo que $M \rightarrow_\beta^* N_1$ y $M \rightarrow_\beta^* N_2$. Por el teorema de Church-Rosser existe N_3 con $N_1 \rightarrow_\beta^* N_3$ y $N_2 \rightarrow_\beta^* N_3$. Pero N_1 y N_2 están en forma normal, luego no contienen redex alguno y no pueden reducirse en pasos no triviales, de donde $N_1 =_\alpha N_3 =_\alpha N_2$.*

El interés de este corolario es doble. Por un lado, justifica que hablemos de *la* forma normal de un término, en singular, cuando esta existe. Por otro, garantiza que el valor de un cómputo, entendido como su forma normal, queda determinado por el término de partida y no por la estrategia de evaluación escogida, de modo que el orden en que decidamos contraer los redexes solo puede afectar a si terminamos, mas no al resultado al que llegamos.

La confluencia se representa mediante el siguiente diagrama, cuya forma de rombo explica que la propiedad reciba también el nombre de *propiedad del diamante*. El vértice superior es el término de partida M, las dos diagonales descendentes son las reducciones divergentes hacia los terminos N_1 y N_2, y las diagonales punteadas representan los pasos que aún restan, garantizados por el teorema, para reconciliar ambas ramas en un término común N_3.



Ejemplo 2.5. Considérese el término $M \equiv (\lambda x.\, x\, x)\, ((\lambda y.\, y)\, z)$, que presenta dos redexes: el externo, que aplica $\lambda x.\, x\, x$ a su argumento, y el interno, $(\lambda y.\, y)\, z$. Contrayendo primero el redex externo, el argumento se duplica y obtenemos $N_1 \equiv ((\lambda y.\, y)\, z)\, ((\lambda y.\, y)\, z)$, que aún requiere dos pasos para resolver las dos copias del redex interno. Contrayendo primero el redex interno, en cambio, llegamos en un solo paso a $N_2 \equiv (\lambda x.\, x\, x)\, z$. Ambas ramas convergen en $z\, z$:

Obsérvese cómo la rama izquierda necesita más de un paso para alcanzar la confluencia, lo cual ilustra por qué el teorema se enuncia sobre $\to_\beta^*$ y no sobre la reducción en un único paso.

De aquí emerge un problema de naturaleza distinta: la terminación. La terminación determina si el cómputo de un término concluye o no. En el λ-cálculo, una secuencia de reducciones puede no detenerse, entrando en un ciclo infinito de pasos que impide alcanzar forma normal alguna, como ilustra el siguiente ejemplo.

Ejemplo 2.6. El ejemplo canónico de término sin forma normal es el combinador $\Omega \equiv (\lambda x.\, x\, x)\, (\lambda x.\, x\, x)$. Al contraer su único redex, el argumento $\lambda x.\, x\, x$ se sustituye en el cuerpo $x\, x$, devolviendo de nuevo Ω:

$$(\lambda x.\, x\, x)\, (\lambda x.\, x\, x) \to_\beta (x\, x)[x := \lambda x.\, x\, x] = (\lambda x.\, x\, x)\, (\lambda x.\, x\, x) \equiv \Omega.$$

La reducción se cierra pues, sobre sí misma:

$$\Omega \equiv (\lambda x.\, x\, x)\, \overbrace{(\lambda x.\, x\, x)}$$

El término es trivialmente confluente, ya que su única reducción conduce siempre a sí mismo, y sin embargo carece de forma normal. Este ejemplo evidencia que la confluencia y la terminación son propiedades independientes: un sistema puede ser confluente sin ser terminante.

El ejemplo de Ω pone de manifiesto que el λ-cálculo sin tipos no es terminante, pues existen términos que carecen de forma normal y cuya reducción no se detiene jamás. La terminación, entendida como normalización fuerte, se recupera al imponer una disciplina de tipos sobre los términos. En particular, el λ-cálculo simplemente tipado sí es fuertemente normalizante, propiedad que abordaremos en un capitulo posterior, al tratar dicho sistema.(Geuvers, 2014; Pierce, 2002).

2.2. Teoría de Tipos

La teoria de tipos es una rama de las matematicas y la logica que tiene como objeto formalizar no solo terminos matematicos, tales como una variable x o una funcion cualquiera f y las operaciones entre ambas, si no tambien, la idea de que cada termino forma parte o tiene un tipo concreto, es decir, que el tipo de una variable $x : \mathbb{N}$ en el dominio de los numeros naturales, es completamente distinguible al tipo de una funcion $f : \mathbb{N} \to \mathbb{N}$.

Los tipos, pese a que son similares a los conjuntos, son ligeramente distintos. Por conjunto, entiendase ocmo una coleccion de elementos, similares, o no, que poseen una propiedad concreta, de donde es posible extenderlos y razonar sobre mas relaciones de pertenencia y formar, por ejemplo, conjuntos dentro de otros conjuntos y definir que un elemento pertenece a varios conjuntos a la vez, cosa que, en los tipos, no es posible pues, un termino es (pertenece a) un tipo unico. Otra caracteristica que los distingue, es que los conjuntos dan informacion semantica y, los tipos por otro lado, proveen informacion sintactica de los objetos. Por ejemplo, comunmente en la teoria de conjuntos definimos que una relacion de pertenencia como sigue:

$$4 \in \{ n \in \mathbb{N} | x > 5 \} \tag{2.1}$$

En la teoria de tipos, un typing judgement, sigue la siguiente notacion que, a lo largo de la presente tesis estaremos empleando:

$$3 + 1 : \mathbb{N} \tag{2.2}$$

Esto puede interpretarse, desde el lente de teoria de tipos, como $3 + 1$ es un termino en los naturales, o bien, de tipo $\mathbb{N}$.

La nocion de tipos se atribuye a Bertrand Russell, quien desarrollo la teoria de tipos en respuesta a las contradicciones que, para entonces, en la teoria de conjuntos y la logica formal existian, como la paradoja de Russell, donde era posible definir, por ejemplo, el conjunto que contiene a todos los conjuntos que no son miembros de sí mismo (TODO: Desarrollar mas esta parte)(Coquand, 2022).

$$R = \{ x | x \in x \} \tag{2.3}$$

entonces

$$R \in R \iff R \notin R \tag{2.4}$$

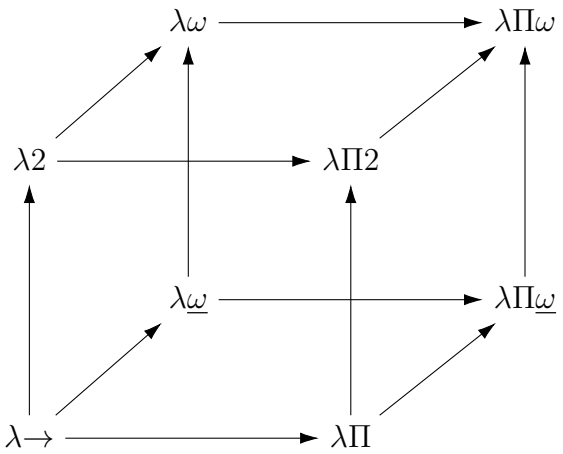
Es, por tanto, de forma contradictoria, un conjunto que es elemento de sí mismo. Para evitar esta paradoja, Russell propuso la distincion entre objetos, predicados, predicados de predicados, etc. Es donde, en esta teoria de tipos, donde la distincion es internalizada por tipos. En esta teoria, a cada objeto se le asigna un tipo, de forma jerargica, de manera tal que conjuntos grandes y pequeños residen en niveles distintos de la teoria.

Los tipos son ubicuos en computación, y numerosos lenguajes tienen su propio sistema de tipos, unos mas avanzados que otros y, otros mas simples. Tan simples que, la manipulacion de tipos es casi perceptible o inextistente. Lenguajes con esta capacidad de omitir tipos son conocidos como lenguajes dinamicamente tipados, donde en casos, es completamente opcional tipar expresiones y en caso contrario, el compilador infiere un tipo adecuado para una expresion. Haskell por ejemplo, es un lenguaje de alto nivel, ampliamente utilizado por grupos de investigacion y academicos, su sistema de tipos esta inicialmente basado en el lambda calculus simplemente tipado, también llamado System F. Hoy en dia su sistema de tipos ha madurado lo suficiente para dejar de considerarle con un sistema de tipos simple. C++ es un claro ejemplo de un sistema de tipos, quiza, mucho mas complejo, pero sumamente rico, que facilita posteriormente, una distincion semantica y rechaza programas que son imposibles de evaluar correctamente. De forma muy general, un sistema de tipos permite al lenguaje alcanzar capacidades de expresividad bastante amplias y, a su vez, evitar que programas errones se ejecuten.

El problema que intenta resolver la teoria de tipos, no es solo encontrar un tipo para un termino concreto, sino tambien, determinar (si lo hay) un termino para un tipo.

-Ejemplo de Gueuvers aca-

A lo largo de los años numerosos esfuerzos se han hecho para desarrollar distintos sabores (sabores es la palabra? quiza extensiones) de la teoria de tipos y poder dar suficiente poder expresivo al lambda calculo. Estos sistemas de tipos fueron explorados por H. Barendregt en el Cubo Lambda (Barendregt, 1991), para investigar las diferentes dimensiones en las que puede extenderse el sistema formal. En su paper describe como cada dimension del cubo representa una dependencia entre terminos y tipos, y sus posibles combinaciones. Entiendase por dependencia con la dependencia mutua, valga la reduncancia, que existe entre tipos y terminos: terminos que dependen de otros terminos, terminos dependientes de tipos, tipos dependientes de terminos, y tipos dependientes de otros tipos. Los ocho vertices del cubo, consecuencia de combinar cada posible dimension, resultan en diferentes sistemas de tipos y expresividad. Concretamente, abordaremos en el presente trabajo, en la extension del vertice inferior derecho, que corresponde a la extension de tipos dependientes, concreamente, λP .



Se empleara, a lo largo de la tesis, la siguiente notacion para definir las derivaciones en la teoria de tipos, donde la parte superior es una lista de premisas y la parte inferior, la conclusion:

$$\frac{\Gamma \vdash A \quad \Gamma, a : A}{\Gamma \vdash a : A} \tag{2.5}$$

Que puede leerse como: si, dado que en el contexto Γ, A es un tipo bien formado y, ademas, en el mismo contexto Γ extendido con $a : A$, entonces podemos inferir que a es de tipo A.

2.2.1. Tipos a la Curry

2.2.2. Tipos a la Church

2.3. Lambda cálculo simplemente tipado

2.4. Tipos dependientes

2.5. Teoria de tipos de Martin-Löf

2.6. Reescritura de terminos en Maude

2.7. Tipos Cociente

Capítulo 3

Un lenguaje dependientemente tipado

3.1. Sintaxis

3.2. Sustitucion

3.3. Normalizacion

3.4. Verificacion de tipos

Capítulo 4

Extension con tipos cociente

4.1. Formacion

4.2. Introduccion

4.3. Eliminacion

4.4. Computacion

Capítulo 5

Conclusiones

Apéndice A

Código Maude

TODO

Bibliografía

Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, revised edition, 1984.

Henk P. Barendregt. *An Introduction To Generalized Type Systmes*. Journal of Functional Porgramming. Catholic University of Nijmegen, Netherlands, revised edition, 1991.

Thierry Coquand. Type Theory. In Edward N. Zalta and Uri Nodelman, editors, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Fall 2022 edition, 2022.

German Geuvers. *Type Theory and Formal Proof*. Cambride University Press, 2014.

Samuel Mimram. *Program = Proof*. Independently published, 2020.

Ulf Norell. *Towards a Practical Programming Language Based on Dependent Type Theory*. PhD thesis, Chalmers University of Technology, 2007.

Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.